CS204 AWS Deepracer Final Report

Weitian Xue

1 Introduction and Objectives

In this project, I trained three AWS DeepRacer models with different speed ranges and reward functions to study the trade-off between lap time and stable completion.

Main goals were:

1. Optimize lap time while keeping high completion stability;
2. Observe the sim-to-real gap by comparing online simulation performance with offline track tests.

All experiments used the same track layout (re:Invent 2018, Counterclockwise) for training, evaluation, and the physical track.

2 Reward Function Design

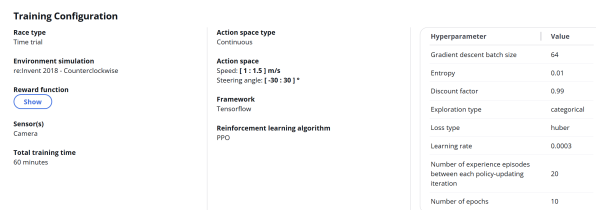
I designed three reward functions (one per model) with increasing aggressiveness from slow to fast. The idea was to start with a stable "stay on track" foundation, then add racing-oriented shaping terms that encourage higher speed and better trajectory control. Full code is included in the Appendix.

2.1 slowModel 1.0-1.5 m/s

This one is designed for more reliability and completion rate for offline test.

Key rewards are:

- Centerline preference and boundary avoidance.
- Mild steering penalty to reduce sudden swerves.
- Emphasize robustness over aggressive speed.



Training Configuration

Race type
Time trial

Environment simulation
re:Invent 2018 - Counterclockwise

Reward function
Show

Sensor(s)
Camera

Total training time
60 minutes

Action space type
Continuous

Action space
Speed: **[1 : 1.5] m/s**
Steering angle: **[-30 : 30] °**

Framework
Tensorflow

Reinforcement learning algorithm
PPO

Hyperparameter	Value
Gradient descent batch size	64
Entropy	0.01
Discount factor	0.99
Exploration type	categorical
Loss type	huber
Learning rate	0.0003
Number of experience episodes between each policy-updating iteration	20
Number of epochs	10

Figure 1: Training configuration screenshot (hyperparameters and setup).

2.2 middleModel 1.3-1.8 m/s

This one is designed to balance speed and stability for better performance in online simulation.

Key rewards are:

- Waypoint-based heading alignment to reduce drifting/oscillation.
- Curvature-aware speed shaping (slower on turns, faster on straights).
- Extra penalties targeting zigzag behaviors.

2.3 fastModel 2.0–4.0 m/s

This one is designed to maximize speed and lap time improvement, accept higher risk.

Key rewards are similar to middleModel, but adjust related settings to achieve the faster speed range.

3 Training Setup and Hyperparameters

All models share the same training setup except for speed range and reward function as shown in Figure 1.

4 Simulation Performance Analysis

I evaluated all models on the same track (re:Invent 2018). Metrics include completion, lap time, and off-track/resets.

4.1 Reward and Completion Trends During Training

Observations:

- slowModel improves smoothly and reaches stable 100% evaluation completion, indicating a consistent policy.
- middleModel achieves strong evaluation reward and good completion in simulation at moderate speed.
- fastModel shows high-variance reward spikes, suggesting occasional excellent runs but reduced consistency.

In RL terms, stronger speed shaping can lead to more fragile policies that work well only under narrow conditions.

4.2 Evaluation Results (Simulation)

Summary (from evaluation screenshots):

- **slowModel (5 trials):** 100% completion; avg lap time $\approx$ 15.07 s; 0 resets/off-tracks.
- **middleModel (5 trials):** 100% completion; avg lap time $\approx$ 12.47 s; 1 reset/off-track.
- **fastModel (3 trials):** 100% completion; avg lap time $\approx$ 11.25 s; 4 resets/off-tracks.

Higher speed reduces lap time but increases instability, especially for fastModel.

5 Physical Performance and Sim-to-Real Gap

I tested slowModel and middleModel on a physical track indoors (no direct sunlight). Tests occurred twice: afternoon (trial) and evening (formal test). The physical track follows the same layout but is assembled from two pieces with a visible cut line.

5.1 Physical Results

slowModel (physical):

- Afternoon: $\sim$5 laps with 0 off-tracks (stable).
- Evening: completed 3 laps with 2 off-tracks; best lap time $\sim$30 s.
- Behavior: noticeable zigzagging; off-tracks often near the same curve and close to the cut line.

middleModel (physical):

- Poor performance in both afternoon and evening; off-track early, despite good simulation evaluation.

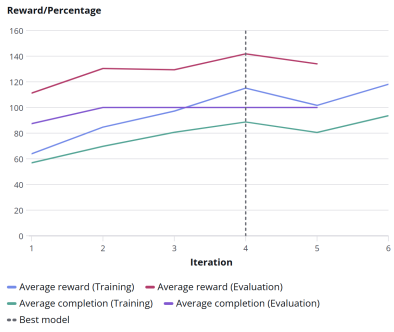
5.2 Sim-to-Real Gap Analysis

Main gap factors:

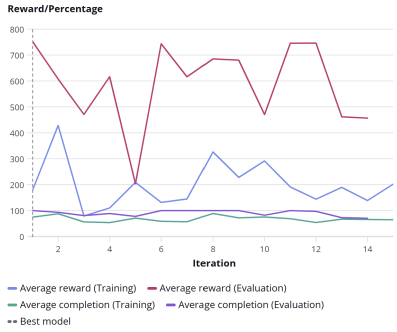
- **Lighting/reflections:** evening indoor lights created reflections that likely affected camera perception and policy decisions.
- **Track seam/cut line:** a visible cut line introduces out-of-distribution visual cues compared to simulation; the car often failed near this region.
- **Robustness vs speed:** middleModel transfers worse than slowModel, suggesting simulation-optimized behavior can be fragile in real conditions.

6 Reflection on Reinforcement Learning Concepts

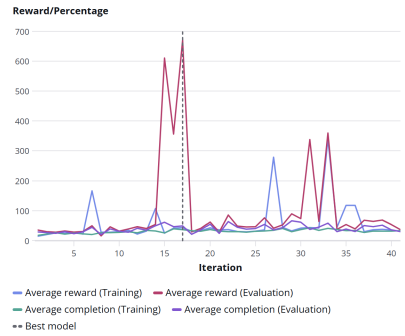
Reward shaping is difficult, especially in the real world. Many intuitive rules (centerline, boundary distance, steering limits) only prove useful after experiments, and their interactions can produce unexpected behaviors. Thus sim-to-real gap is significant: even 100% completion in simulation can produce "weird" actions in reality due to perception shifts.



(a) slowModel



(b) middleModel



(c) fastModel

Figure 2: Reward and completion vs iteration for the three models.

Evaluation results

Trial	Time (MM:SS.mmm)	Trial result (% track completed)	Status	Resets	Off-trac
1	00:14.940	100%	Lap complete	0	0
2	00:15.299	100%	Lap complete	0	0
3	00:15.239	100%	Lap complete	0	0
4	00:14.841	100%	Lap complete	0	0
5	00:15.043	100%	Lap complete	0	0

(a) slowModel

Evaluation results

Trial	Time (MM:SS.mmm)	Trial result (% track completed)	Status	Resets	Off-trac
1	00:11.859	100%	Lap complete	0	0
2	00:12.131	100%	Lap complete	0	0
3	00:12.070	100%	Lap complete	0	0
4	00:12.135	100%	Lap complete	0	0
5	00:14.177	100%	Lap complete	1	1

(b) middleModel

Evaluation results

Trial	Time (MM:SS.mmm)	Trial result (% track completed)	Status	Resets	Off-trac
1	00:10.605	100%	Lap complete	1	1
2	00:10.544	100%	Lap complete	1	1
3	00:12.590	100%	Lap complete	2	2

(c) fastModel

Figure 3: Evaluation result screenshots (same track as training).

If I continued it in the future, I would focus on robustness (reduce sensitivity to visual artifacts), constrain steering more at high speed, and iterate more with physical testing.

7 References

AWS DeepRacer Documentation:
`https://docs.aws.amazon.com/deepracer/`

A Appendix A: Reward Function Code

A.1 A.1 slowModel Reward Function

```
# (keep your slowModel code here exactly as you pasted)
def reward_function(params):
   # read input parameters
   all_wheels_on_track = params["all_wheels_on_track"]
   distance_from_center = params["distance_from_center"]
   track_width = params["track_width"]
   abs_steering = abs(params["steering_angle"])

   # base minimal reward
   if not all_wheels_on_track:
      return 1e-3

   # markers for distance-from-center reward
   marker_1 = 0.10 * track_width
   marker_2 = 0.25 * track_width
   marker_3 = 0.50 * track_width

   # 1) center line reward
   if distance_from_center <= marker_1:
      reward = 1.0
   elif distance_from_center <= marker_2:
      reward = 0.5
   elif distance_from_center <= marker_3:
      reward = 0.1
   else:
      return 1e-3

   # 2) stay inside borders
   if (0.5 * track_width - distance_from_center) < 0.05:
      reward *= 0.2
   else:
      reward *= 1.0

   # 3) steering penalty
   ABS_STEERING_THRESHOLD = 15
   if abs_steering > ABS_STEERING_THRESHOLD:
      reward *= 0.8

   return float(reward)
```

A.2 A.2 middleModel Reward Function

```
# (keep your middleModel code here exactly as you pasted)
import math

def reward_function(params):
   all_wheels_on_track = params["all_wheels_on_track"]
   is_offtrack = params.get("is_offtrack", False)
   is_reversed = params.get("is_reversed", False)

   track_width = params["track_width"]
   distance_from_center = params["distance_from_center"]
   heading = params["heading"]
```

```
speed = params["speed"]
steering = abs(params["steering_angle"])
progress = params["progress"]
steps = params["steps"]
waypoints = params["waypoints"]
closest_waypoints = params["closest_waypoints"]

if is_offtrack or (not all_wheels_on_track) or is_reversed:
   return 1e-3

def direction_between(p1, p2):
   return math.degrees(math.atan2(p2[1] - p1[1], p2[0] - p1[0]))

def angle_diff(a, b):
   d = abs(a - b) % 360
   if d > 180:
      d = 360 - d
   return d

prev_wp = waypoints[closest_waypoints[0]]
next_wp = waypoints[closest_waypoints[1]]

lookahead = 5
next2_idx = (closest_waypoints[1] + lookahead) % len(waypoints)
next2_wp = waypoints[next2_idx]

direction_now = direction_between(prev_wp, next_wp)
direction_ahead = direction_between(next_wp, next2_wp)
smoothed_direction = direction_between(prev_wp, next2_wp)

track_direction = smoothed_direction
direction_error = angle_diff(track_direction, heading)

future_direction = direction_ahead
turn_angle = angle_diff(direction_now, future_direction)

reward = 1.0

half_width = track_width / 2.0
center_ratio = distance_from_center / half_width
center_factor = max(1e-3, 1.0 - (center_ratio ** 2))
reward *= center_factor

if direction_error > 30:
   return 1e-3
heading_factor = max(1e-3, 1.0 - direction_error / 30.0)
reward *= (0.5 + 0.5 * heading_factor)

if turn_angle < 6:
   target_speed = 1.8
elif turn_angle < 12:
   target_speed = 1.65
elif turn_angle < 20:
   target_speed = 1.5
else:
   target_speed = 1.35

speed_diff = abs(speed - target_speed)
```

```
    if speed_diff < 0.05:
        speed_factor = 1.3
    elif speed_diff < 0.10:
        speed_factor = 1.1
    elif speed_diff < 0.20:
        speed_factor = 0.85
    else:
        speed_factor = 0.6

    reward *= speed_factor

    if turn_angle < 10 and steering > 10:
        reward *= 0.5
    if turn_angle < 10 and steering > 15:
        reward *= 0.3

    if steering > 25:
        reward *= 0.7
    if speed > 1.65 and steering > 12:
        reward *= 0.7

    pace = progress / max(steps, 1)
    reward += 0.8 * pace

    if progress > 99.5:
        finish_bonus = max(50.0, 500.0 - steps)
        reward += finish_bonus

    return float(max(reward, 1e-3))
```

A.3 A.3 fastModel Reward Function

```
# (keep your fastModel code here exactly as you pasted)
import math

def reward_function(params):
    all_wheels_on_track = params["all_wheels_on_track"]
    is_offtrack = params.get("is_offtrack", False)
    is_reversed = params.get("is_reversed", False)

    track_width = params["track_width"]
    distance_from_center = params["distance_from_center"]
    heading = params["heading"]
    speed = params["speed"]
    steering = abs(params["steering_angle"])
    progress = params["progress"]
    steps = params["steps"]
    waypoints = params["waypoints"]
    closest_waypoints = params["closest_waypoints"]

    if is_offtrack or (not all_wheels_on_track) or is_reversed:
        return 1e-3

    def direction_between(p1, p2):
        return math.degrees(math.atan2(p2[1] - p1[1], p2[0] - p1[0]))

    def angle_diff(a, b):
```

```
    d = abs(a - b) % 360
    if d > 180:
        d = 360 - d
    return d

prev_wp = waypoints[closest_waypoints[0]]
next_wp = waypoints[closest_waypoints[1]]

lookahead = 6
next2_idx = (closest_waypoints[1] + lookahead) % len(waypoints)
next2_wp = waypoints[next2_idx]

direction_now = direction_between(prev_wp, next_wp)
direction_ahead = direction_between(next_wp, next2_wp)
smoothed_direction = direction_between(prev_wp, next2_wp)

track_direction = smoothed_direction
direction_error = angle_diff(track_direction, heading)

future_direction = direction_ahead
turn_angle = angle_diff(direction_now, future_direction)

reward = 1.0

half_width = track_width / 2.0
center_ratio = distance_from_center / half_width
center_factor = max(1e-3, 1.0 - (center_ratio ** 2))
reward *= center_factor

if direction_error > 25:
    return 1e-3
heading_factor = max(1e-3, 1.0 - direction_error / 25.0)
reward *= (0.35 + 0.65 * heading_factor)

if turn_angle < 4:
    target_speed = 4.0
elif turn_angle < 8:
    target_speed = 3.6
elif turn_angle < 14:
    target_speed = 3.2
elif turn_angle < 22:
    target_speed = 2.8
else:
    target_speed = 2.4

speed_diff = abs(speed - target_speed)

if speed_diff < 0.20:
    speed_factor = 1.60
elif speed_diff < 0.45:
    speed_factor = 1.25
elif speed_diff < 0.80:
    speed_factor = 0.90
else:
    speed_factor = 0.55

reward *= speed_factor

if turn_angle < 6:
```

```
        reward *= (1.0 + 0.45 * (speed / 4.0))

    if turn_angle < 8 and steering > 6:
        reward *= 0.35
    if turn_angle < 8 and steering > 10:
        reward *= 0.15
    if turn_angle < 14 and steering > 14:
        reward *= 0.45

    if steering > 25:
        reward *= 0.60
    if speed > 3.2 and steering > 9:
        reward *= 0.55
    if speed > 3.6 and steering > 7:
        reward *= 0.50

    pace = progress / max(steps, 1)
    reward += 2.2 * pace

    if progress > 99.5:
        finish_bonus = max(120.0, 3000.0 - 14.0 * steps)
        reward += finish_bonus

    return float(max(reward, 1e-3))
```